Departamento de Ciencia de la Computación e Inteligencia Artificial
Universidad de Alicante **Curso 2026/2027**

Práctica 2. Búsqueda y Satisfacción de Restricciones.

Descripción general

En esta práctica vamos a aplicar técnicas de inteligencia artificial para la resolución del conocido pasatiempos Sudoku. Veremos que aunque el problema puede solucionarse con métodos algorítmicos de búsqueda por fuerza bruta con el algoritmo de back propagation, utilizando técnicas de satisfacción de restricciones podemos resolver el problema mediante una búsqueda inteligente con lo que ahorramos en memoria utilizada y tiempo de cómputo.

El Sudoku, en su versión básica, consiste en un tablero de tamaño 9x9 en el que el jugador ha de insertar números del 1 al 9. Las restricciones del juego consisten en que no se puede repetir el mismo número en una fila o en una columna del tablero. Asimismo, tampoco se pueden repetir números en cada uno de los 9 subtableros de tamaño 3x3.

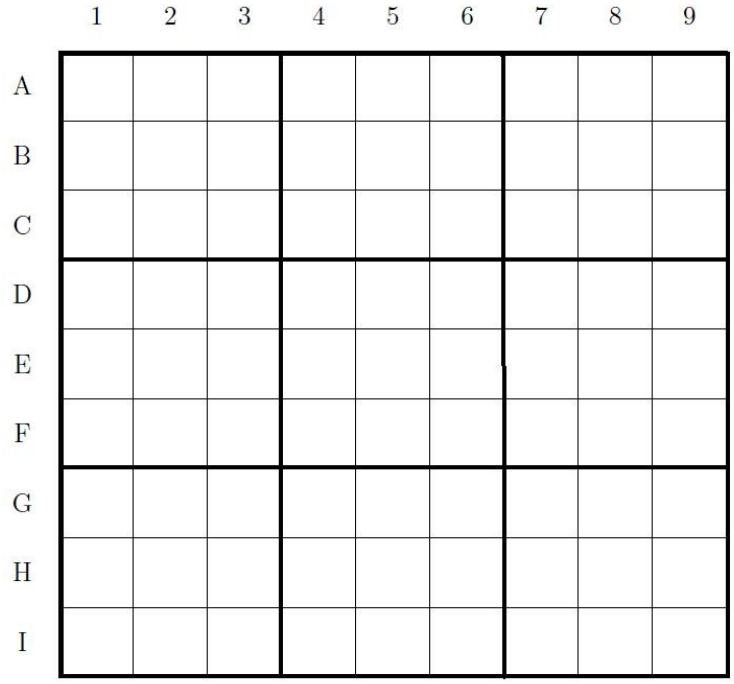


Ilustración 1: Tablero de sudoku. Se pueden observar los subtableros 3x3

Para empezar el sudoku, se dan una serie de números iniciales y el resto de casillas se dejan vacías. La dinámica del juego consiste en ir rellenando todas las casillas cumpliendo las reglas establecidas. Debemos ir colocando números del 1 al 9 en cada casilla vacía.

Introducción y entorno de trabajo. Diseño del algoritmo de búsqueda de la solución por fuerza bruta.

. Python

Para el desarrollo de la práctica se utilizará Python como lenguaje de programación. Se proporcionará un proyecto Python con el código necesario para lanzar un entorno gráfico que permita visualizar diferentes tableros de SuDoKu y ejecutar los distintos algoritmos que el alumno ha de implementar para superar la práctica.

. Proyecto

El proyecto contiene las siguientes 3 clases:

- **main.py** Esta clase se encarga de crear el entorno gráfico y gestionar los eventos ocasionados por la interacción con el usuario.
- **tablero.py** Esta clase mantiene la representación del tablero de SuDoKu. Para realizar los diferentes algoritmos de búsqueda recibiremos un objeto de esta clase con el estado actual del tablero. Contiene los métodos para obtener información de cada casilla y para su modificación. También permite comprobar si se cumplen las restricciones del juego.
- **solver.py**. Clase donde se implementarán los métodos de búsqueda requeridos en esta práctica.

Las clases **main.py y tablero.py no podrán ser modificadas** por el alumno. El alumno modificará únicamente los métodos incluidos en la **clase Solver**. Se podrán añadir a esta clase las propiedades y métodos adicionales que el alumno estime convenientes. Asimismo, se podrán añadir clases auxiliares si fuese necesario.

. Aplicación

Tras compilar y ejecutar el proyecto proporcionado dará comienzo una aplicación con un interfaz de usuario (UI) como el que se muestra en la Ilustración 2. En la ventana que forma la UI encontraremos el tablero donde se representará el juego del sudoku junto a una serie de componentes para lanzar las distintas acciones.

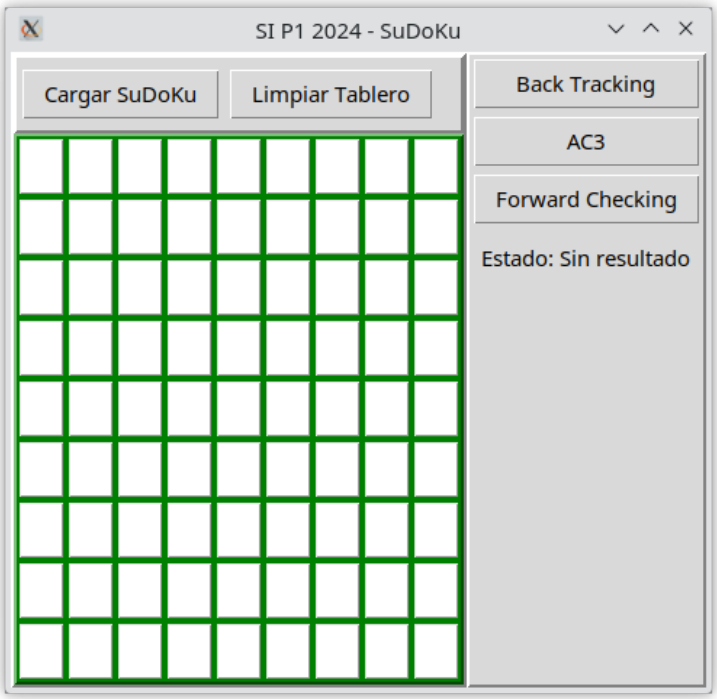


Ilustración 2: Interfaz de usuario de la aplicación Sudoku

En la parte superior encontramos dos botones. El primero nos permitirá cargar un sudoku inicial desde un fichero de texto plano. El segundo nos permitirá borrar todo el tablero. A la derecha del tablero encontramos los controles que nos van a permitir seleccionar el método automático de resolución: BackTracking, AC3 o Fordward Checking. El estado del resultado de la ejecución de cada algoritmo se mostrará en la parte inferior de la ventana.

Cono se ha comentado, es posible cargar el contenido de las casillas del sudoku desde un archivo de texto plano. Junto con el código de la práctica se han dejado varios ficheros que pueden ser utilizados de ejemplo. El formato del fichero es simple. Contendrá los 81 valores del tablero. Si se quiere dejar una casilla vacía, bastará con poner 0 en su posición.

Implementación del algoritmo backtraking (Obligatorio, hasta 2 puntos).

En primer lugar se implementará el algoritmo de backtracking para encontrar la solución a cada sudoku propuesto. El algoritmo comenzará a ejecutarse mediante una llamada al método *ejecutarBackTracking* de la clase Solver. Este método recibe un objeto de la clase Tablero con el que trabajará. El objetivo será encontrar una solución al tablero recibido. El algoritmo ha de rellenar el tablero sin alterar los valores presentes en el tablero al comienzo del mismo (exceptuando, claro, las casillas que contienen un 0 al ser considerado este valor como de casilla vacía).

```
algoritmo BT(int k, V[N]): bool
     si k>N entonces:
          devolver TRUE
     sino:
          Para cada dk ∈ Dk hacer:
               V[k] = dk
               si esCorrecto(V) entonces:
                    si BT(k+1, V) entonces:
                         devuelve TRUE
                    finSi
               finSi
          finPara
          devuelve FALSE
     FinSi
FinAlgoritmo
```

Backtracking hará uso de la reducción de dominios efectuada por AC3 en caso de que hayamos ejecutado este algoritmo primero (OBLIGATORIO).

Es importante mostrar por la salida estándar información sobre el proceso de búsqueda que nos pueda orientar sobre el coste de encontrar la solución mediante este método.

Implementación del algoritmo AC3 (Obligatorio, hasta 5 puntos)

El algoritmo AC3 reduce los dominios de las variables eliminado aquellos valores que seguro no van a formar parte de la solución:

```
Q = {c(ep) = <Vi, Vj>|ep ∈ E, i ≠ j}
Mientras Q ≠ Ø hacer
     <Vk, Vm> = seleccionar_y_borrar(Q)
     cambio = falso
     Para todo vk  ∈  Dk hacer
          Si no_consistente (vk, Dm) entonces:
               borrar (vk, Dk)
               cambio = cierto
          FinSi
     FinPara
     Si Dk = Ø entonces: salir_sin_solución FinSi
     Si cambio = cierto entonces:
          Q = Q ∪ {c(er) = <Vi, Vk> | er ∈ E, i ≠ k, i ≠ m}
     FinSi
FinMientras
```

Es importante mostrar por la salida estándar información sobre el proceso de búsqueda que nos pueda orientar sobre el coste de encontrar la solución mediante este método.

Implementación del algoritmo Forward Checking (Optativo, hasta 3 puntos)

Se implementará también el método Forward Checking visto en clase.

Es importante mostrar por la salida estándar información sobre el proceso de búsqueda que nos pueda orientar sobre el coste de encontrar la solución mediante este método: tiempo de ejecución, saltos atrás/veces que se ejecuta la función restaurar, etc.

Evaluación

La nota de la práctica se calcula como la suma de las notas obtenidas en las implementaciones de los apartados 3, 4 y 5 según la puntuación indicada en cada apartado. Los apartados 3 y 4 son obligatorios lo que implica que deben estar implementados, documentados y funcionar correctamente. En caso contrario la práctica se considerará suspensa independientemente de la nota obtenida en el apartado 5.

. Documentación

Es obligatorio documentar todo el trabajo realizado. La documentación debe estar orientada a comprobar y comprender el funcionamiento de cada algoritmo. Para ello será necesario diseñar una serie de pruebas que permitan ver, por un lado, que los algoritmos funcionan correctamente, y por el otro, las diferencias de funcionamiento entre algoritmos. La documentación se realizará en un fichero en formato PDF.

. Entrega

La entrega de la práctica se realizará a través de la página Moodle de la asignatura. La fecha de entrega queda fijada para el **3 de enero de 2027 a las 23.55 horas**.

Para realizar la entrega se comprimirá todo el proyecto (haremos Clean antes de comprimirlo) junto con el fichero de documentación. El nombre del fichero comprimido tendrá el siguiente formato: "`Apellido1Apellido2Nombre.tgz`". Un fichero de ejemplo sería `ViejoHernandoDiego.tgz` (podemos usar cualquier formato de compresión de entre los habituales).

- ¡¡¡IMPORTANTE!!! no cumplir cualquiera de las normas de formato/entrega anteriores puede suponer un suspenso en la práctica.
- ¡¡¡IMPORTANTE!!! Recordad que las prácticas son individuales y NO se pueden hacer en parejas o grupos. Cualquier código copiado supondrá un suspenso de la práctica para todas las personas implicadas en la copia. Se utilizarán herramientas para la detección de copia. Estos hechos serán comunicados a la Escuela Politécnica para que se tomen las medidas disciplinarias oportunas contra los infractores.